

Esercizio 2 (11 punti).

Considerare l'architettura RISC-V a ciclo singolo nella figura in basso (e in allegato).

Si vuole aggiungere alla CPU l'istruzione **jump and link if divisible by 4** (**jalf**), di **tipo I** e sintassi assembly:

jalf rd, rs1, immediate

L'istruzione deve operare come segue:

- carica dal banco registri **rs1**;
- estende di segno il campo immediato **immediate** (si può usare **Genera cost** per questa operazione). Chiameremo questa word **immediate₃₂**;
- se **immediate₃₂** è divisibile per 4, salva il valore di **PC + 4** nel registro **rd**;
- se **immediate₃₂** è divisibile per 4, aggiorna il PC con il valore **rs1 + immediate₃₂**.

Esempio: Supponiamo che **rd** sia **a0**, che **rs1** sia **t4** contenente il valore 3, e che **immediate** valga **0xFC = 0b000011111100 = 252**

In tal caso:

- 0xFC è divisibile per 4, quindi il valore del PC + 4 verrà salvato in **a0**.
- 0xFC è divisibile per 4, quindi la parte immediata verrà estesa di segno (rimanendo 252 in questo caso), e sommata a **t4**.
- Tale valore verrà infine scritto nel PC, che conterrà **252+3 = 255** alla fine dell'istruzione.

- Mostrare le **modifiche all'architettura** della CPU RISC-V, avendo cura di aggiungere eventuali altri componenti necessari a realizzare l'istruzione. A tal fine, si può alterare la stampa del diagramma architetturale oppure ridisegnare i componenti interessati dalla modifica, avendo cura di indicare i fili di collegamento e tutti i segnali entranti ed uscenti. Indicare inoltre sul diagramma i **segnali di controllo** che la CU genera *per realizzare l'istruzione*.
- Indicare il contenuto in bit della word che esprime l'istruzione

jalf t4, s4, 0x203

compilando la tabella sottostante (assumiamo che lo **OpCode** di **jalf** sia **0x3D**, e che porzioni inutilizzate dall'istruzione siano codificate con zeri).

0	0	1	0	0	0	0	0	0	0	0	1	1	1	0	1	0	0	0	0	0	1	1	1	0	1	0	1	1	1	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- Supponendo che l'accesso alle **memorie** impieghi **200 ns**, l'accesso ai **registri** **50 ns**, le operazioni dell'**ALU** e dei **sommatori** **150 ns**, e che gli altri ritardi di propagazione dei segnali siano trascurabili, indicare la durata totale del **ciclo di clock** tale che anche l'esecuzione della nuova istruzione sia permessa *spiegando i calcoli effettuati*.
- Indicando con **jalf** il segnale di controllo che viene asserito per eseguire la nuova istruzione, assumiamo che
 - tutti i segnali di tipo *don't care* siano pari a 0 e che
 - la Control Unit della CPU RISC-V modificata per supportare **jalf** sia difettosa e sovrascriva il segnale **RegWrite** come segue:
RegWrite = MemWrite (il simbolo = denota che la variabile a sinistra assume il valore dato della variabile a destra)
 In tal caso, indicare quale valore sia assegnato a **a7** al termine dell'esecuzione del seguente frammento di codice, assumendo che i registri **a7**, **s0** e **s1** siano inizializzati a 0. Motivare la propria risposta.

addi s0, s0, 0x4

sw t0, 0(s0)

beq s1, s0, Out

jalf x0, Exit

Out: add a7, zero, s1

Exit: addi a7, a7, 0x8002

L'ultima istruzione non è valida perché la parte immediata è più di 12 bit quindi ci sarebbe un'eccezione all'ultima istruzione. Nessuna istruzione scrive nel banco dei registri, quelle di tipo R e I perché RegWrite è erroneamente a zero, e la sw perché ha RegWrite erroneamente a 1 ma la sezione corrispondente a rd nell'istruzione corrisponde ai primi 5 bit della parte immediata, e quindi non scrive (cioè "scrive" in x0).

